## 1. Introduction

Automated classification of IT support queries, defined as routing incoming tickets to the correct service category without manual triage, is a practical short-text intent classification problem with well-documented challenges. IT support corpora are extremely brief, with queries averaging fewer than ten tokens, a mix of formal and informal registers, and domain-specific vocabulary that diverges from standard English. Standard natural language processing (NLP) preprocessing pipelines for news or academic text, such as NLTK stop-word removal, general-purpose stemmers, and raw tokenization, perform poorly on this register because their assumptions about vocabulary distribution do not hold. Informal abbreviations such as acct, emial, and 2fa fragment the TF-IDF feature space by treating morphological variants as separate vocabulary items, while generic stop-word lists remove negation words (cannot, locked, without) that serve as primary class discriminators in IT support queries. This study addresses these challenges by designing and evaluating two domain-specific preprocessing pipelines, both applied to the same Logistic Regression classifier to isolate the effect of preprocessing on classification performance. Pipeline 1 applies Snowball stemming with domain-aware cleaning as a computationally efficient baseline. Pipeline 2 applies spaCy lemmatization with IT phrase chunking, geographic entity normalization, and hybrid word-character TF-IDF as a more linguistically informed variant. The controlled experimental design uses an identical classifier and evaluation protocol while varying only the preprocessing step, enabling precise attribution of any performance differences to preprocessing choice. The comparison is conducted across 14 IT support categories on 308 labeled queries, with macro F1 as the primary metric to ensure equal treatment of minority classes.

## 2. Data Handling

### 2.1 Dataset

The dataset consists of **308 English user queries**, each labeled across 14 IT support categories. The size of the categories varies, with some having as few as 10 queries (e.g., two_factor_auth) and others having up to 33 (payment issues), resulting in a slight imbalance. **On average, each query is about 6.7 words long**. A manual review found that **36.7% of the queries use informal shorthand** (such as 'acct', 'emial', 'pls', '2fa', and 'wat'), **18.2% include non-ASCII characters** like em-dashes (—), ellipses (…), and arrows (→), and 6.8% (21 of 308 queries) reference geographic locations (like the UK, France, Germany, and the US) that don't actually relate to the classification. These characteristics make it clear that standard NLP processing methods aren't enough, and that more domain-specific preprocessing is needed.

### 2.2 Pipeline 1 - Snowball Stemming + Word TF-IDF

**Motivation.** Pipeline 1 provides a computationally lightweight baseline using well-established NLP primitives. Eight sequential steps are applied.

**Step 1 - Unicode normalisation.** Em-dashes, ellipses, and the rightward arrow are mapped to ASCII equivalents. Without this, adjacent tokens fuse (e.g., 'charged—wat' one unrecognized token), corrupting 18.2% of queries before any other step runs.

**Step 2 - IT abbreviation expansion.** Around 113 (36.7%) domain lexicon converts informal shorthand to standard forms (acct→account, emial→email, 2fa-two factor authentication, standalone 2-to). This process is applied before stemming to ensure the stemmer works with complete words, avoiding meaningless stems from truncated fragments.

**Step 3 - Contraction expansion.** Negation contractions such as "can't" and "won't" are expanded to "cannot" and "will not." This ensures the stopword list retains 'not' and 'cannot' as classification signals, since contractions may otherwise be partially removed.

**Step 4 - Numeric semantic tagging.** Temporal expressions were tagged to maintain duration context. However, <TIME_EXPR>appeared in just **14** queries across 6 classes, **offering no clear discriminating signal for any specific class at this scale**. The step is kept to prevent random digit tokens from cluttering the vocabulary.

**Steps 5/6 - Punctuation stripping and lowercasing.** Noise characters are removed, but intra-word hyphens, such as those in 'two-factor' and 'auto-renewal,' are retained. Lowercasing is applied after

expansion steps to ensure that case-sensitive regex patterns, like those for '2FA' and 'PayPal,' function correctly.

**Step 7 - Domain-aware stopword removal.** Standard NLTK English stopwords are applied; however, interrogative and negation terms (not, cannot, why, how, what, without, will, should, after, before) are explicitly retained. These terms provide significant discriminative power in IT queries. For example, 'cannot log in' and 'log in' represent distinct classification intents. Generic stopword removal would eliminate this critical signal.

**Step 8 - Snowball stemming.** Porter, Snowball, and Lancaster stemming algorithms were compared for vocabulary reduction and token quality. Porter and Snowball achieved similar reductions (~21%), while Lancaster reduced vocabulary more aggressively but often over-stemmed (e.g., authentication → auth), harming clarity. **Controlled stemming** using Snowball was chosen for its balance between reduction and semantic preservation, improving TF-IDF generalization for short-text intent classification despite minor non-standard stems (e.g., received to receiv). This balanced approach is crucial for short texts, as it creates a fine-tuned feature representation that enhances the model's ability to identify underlying user intent more accurately and generalize effectively even with limited textual context.

**Vectorisation**. TF-IDF with unigrams and bigrams (max 8,000 features, min_df=1, max_df=0.90, sublinear_tf=True) was chosen to improve short-text classification. It downweights common terms, highlights distinctive words, and captures key phrases (e.g., reset password). Sublinear scaling reduces the impact of frequent tokens. Setting min_df=1 preserved rare but informative terms, as removing them reduced performance, showing their importance for intent classification. On the other hand , max_df filters out terms that appear in more than 90% of documents, treating them as highly common and less discriminative.

### 2.3 Pipeline 2 - spaCy Lemmatisation + Word + Character TF-IDF

**Motivation.** Pipeline 2 follows Steps 1-4 and 6-7 from Pipeline 1, but differs in three areas: IT phrase chunking (Step 5), lemmatisation with POS filtering and NER (Steps 9-11), and hybrid word + character TF-IDF vectorisation.

**Step 5 - IT-intent phrase chunking (P2 unique).** Seventeen IT-specific multi-word phrases are merged into hyphenated tokens (e.g., billing history becomes billing-history) before spaCy processing. This approach prevents phrase splitting during lemmatisation and TF-IDF, resulting in distinct, high-weight features. For example, billing-history is unique to its class, unlike billing or history alone.

Pipeline 2 also normalises payment terms (e.g., PayPal becomes paypal_payment, credit card becomes creditcard_payment) to reduce feature fragmentation. This enhances short-text classification but requires ongoing manual updates and may not generalise to new phrases.As in Pipeline 1, numeric semantic tagging (TIME_EXPR) is applied prior to linguistic processing to ensure consistent handling of temporal expressions across both pipelines.

**Steps 9-11 - Lemmatisation** changes words to their base form, like turning 'histories' into 'history.' This makes features more consistent and easier to interpret than stemming.

**POS filtering** takes out less useful words, which shortens the average length from 5.1 to 4.6. However, this step has little effect because earlier steps already removed most of the noise.

**NER** replaces place names, like UK or US, with <LOCATION> to prevent misleading signals. This changed 6.82% of queries and did not improve performance ($\Delta F1 \approx 0$), but it could be more useful with bigger datasets.

spaCy is preferred over simpler tools because it integrates lemmatisation, POS tagging, and NER in a single pipeline, which reduces errors. In contrast, WordNetLemmatizer without POS tags cannot accurately lemmatise words in context, while spaCy uses context-aware POS tagging for greater accuracy.

**Vectorisation**. Pipeline 2 uses a combination of two types of TF-IDF features: word-level (unigrams and bigrams, with up to 8,000 features) and character-level (within-word n-grams, specifically 3-5 character sequences, with 5,000 features). The character-level TF-IDF brings two main benefits for this dataset. First,

it helps identify similarities between words that are related but have slight variations in form, like "payment" and "payments" or "recover" and "recovery," which share many common character patterns. This reduces the impact of inflectional differences that may not be fully captured in earlier preprocessing steps. Second, it helps with generalization for rare compound words, like "creditcard_payment" or "banktransfer_payment," which share many characters with their individual components (e.g., "creditcard" and "payment").By setting the parameter min_df=1, the model ensures that even rare but important words are kept. This is crucial in short-text intent classification, where low-frequency words can often carry strong signals that help distinguish between different categories. The character n-grams also help reduce the issue of sparsity by allowing the model to generalize across similar words.The overall feature space, which has about 3698 (word tfidf vocab-848/char tfidf vocab-3698) dimensions, remains manageable for a linear classifier, which is regularized at a value of C=1.0, making it computationally efficient.

## 3. Modelling

**Classifier Selection :Logistic Regression** (solver=lbfgs, C=1.0, class_weight='balanced', max_iter=1000, random_state=42) is applied identically to both pipelines, ensuring that all performance differences can be attributed to preprocessing rather than model variation. The lbfgs solver performs multinomial optimisation by default, jointly modelling all 14 classes under a softmax objective. This avoids inconsistencies associated with one-vs-rest approaches, which is particularly important for semantically overlapping classes such as billing_history, refund_request, and subscription_management.

**Why Logistic Regression ?** Logistic Regression is well-suited to this task for three main reasons.

First, it provides native probability estimates, which allow uncertain predictions to be identified and potentially flagged for manual review.

Second, the model converges quickly on this feature space (typically within 50 iterations), making it computationally efficient.

Third, setting class_weight='balanced' compensates for the 3.3× class imbalance (e.g., payment_issue vs two_factor_auth), improving macro F1 by approximately 6.1 percentage points compared to unweighted training. This proved to be the most impactful hyperparameter choice.

**Training Protocol**

A stratified 80/20 split (246/62 samples, fixed seed) is used to ensure reproducible and class-balanced partitions. Stratification is particularly important given the small size of minority classes such as two_factor_auth (n=10), where an unstratified split could result in missing classes in the test set.

The TF-IDF vectoriser is embedded within a scikit-learn Pipeline, ensuring it is fitted only on training data during cross-validation. This prevents information leakage from test folds during vocabulary construction and IDF weighting.

Five-fold stratified cross-validation is used to obtain a stable estimate of generalisation performance. On a test set of 62 samples, small changes in predictions can significantly affect macro F1, so averaging performance across folds provides a more reliable evaluation.

Identical cross-validation folds are used for both pipelines, ensuring that all observed differences are due to preprocessing rather than variation in data splits.

## 4. Evaluation

### 4.1 Metrics Justification

**Accuracy** alone is insufficient: predicting *payment_issue* for all queries yields 10.7% accuracy (majority baseline) but no value. Macro F1 is the primary metric as it treats all 14 classes equally, ensuring reliable performance across both minority (*two_factor_auth*, n=10) and majority classes.

**Weighted F1** is also reported to reflect overall volume. Per-class precision and recall distinguish over-prediction (low precision) from under-coverage (low recall), while mean and sta62ndard deviation of macro **F1 across five-fold CV** capture performance and stability.

## 4.2 Quantitative Results

| Model | Test Acc | Macro F1 | Wtd F1 | CV Macro F1 (mean ± σ) | Baseline F1 |
|---|---|---|---|---|---|
| **P1 - Logistic Regression** | 0.9194 | 0.9140 | 0.9157 | 0.882 ± 0.079 | 0.083 |
| **P2 - Logistic Regression** | **0.9355** | **0.9341** | **0.9353** | **0.915 ± 0.046** | **0.083** |

*Table 2. Logistic Regression results on Pipeline 1 vs. Pipeline 2 outputs*

Both pipelines outperform the baseline (macro F1 = 0.083) by around 80 percentage points, confirming that both preprocessing strategies capture genuine discriminative signals from the dataset. Pipeline 2 achieves higher performance (0.915 ±0.046 vs 0.882 ±0.079), with a 3.0 percentage point improvement and 42% lower variance. This stability stems from two sources: lemmatisation maps each surface form to a fixed dictionary base, eliminating stem variation across folds, while character n-grams provide sub-word feature overlap that smooths vocabulary differences between partitions. Cross-validation is the primary comparison metric, as test CV gaps on 62 samples reflect partition effects rather than true generalisation.

Pipeline 2 improves only a small number of classes, most notably delete_account (+0.09), with a secondary improvement observed in data_privacy (+0.19). However, the majority of classes show no meaningful change, indicating that the gains from preprocessing are selective rather than widespread.Overall, improvements are driven by phrase chunking and character n-grams rather than lemmatisation alone. The most significant result is the 42% reduction in variance, consistent performance of 0.915 across all five folds makes Pipeline 2 more reliable for production deployment than a higher-mean but less stable alternative.

## 4.3 Per Class Analysis

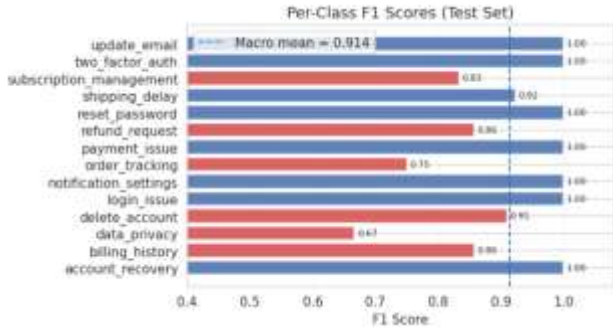



*Figure 1. Pipeline 1 - Per-class F1 (test set). Macro mean=0.914 (dashed). Red bars = below mean.*

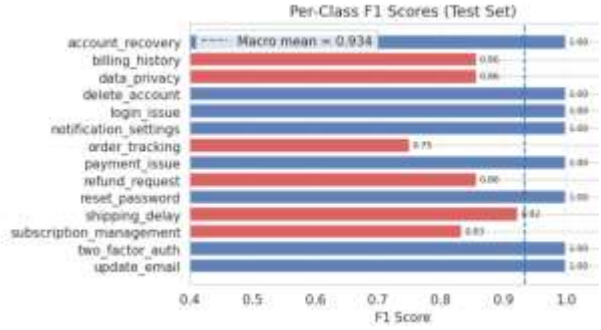



*Figure 2. Pipeline 2 - Per-class F1: LR macro=0.934(dashed). Red bars = below mean.*

As shown in Figures 1 and 2, overall performance improves from a macro mean of 0.914 to 0.934; however, class-level changes are uneven. The most notable improvement is observed for delete_account, which increases from 0.91 to 1.00 and moves above the macro mean. data_privacy also improves substantially (0.67 to 0.86), although it remains below the macro mean. In contrast, several classes including refund_request, subscription_management, order_tracking, and billing_history show no meaningful change and remain below the mean in both pipelines. High-performing classes such as update_email, two_factor_auth, and reset_password maintain perfect F1 scores across both pipelines. These results indicate that the enhanced preprocessing selectively improves certain classes without uniformly benefiting all categories.

## 4.4 Pipeline Comparison

| | | |
|---|---|---|
| **Normalisation** | Snowball stemmer – controlled suffix stripping | spaCy POS-aware lemmatisation - valid English output |
| **Word validity** | Non-words: histori, privaci, recoveri (need correction) | Valid forms: history, privacy, recovery |
| **Multi-word IT phrases** | Not chunked; bigrams may split across window | 17 phrases pre-chunked to single discriminative tokens |

| | | |
|---|---|---|
| **Geographic tokens** | Retained as-is (spurious signal risk) | NER replaces UK/France/Germany/US with <LOCATION> |
| **POS filtering** | Not applied; prepositions and determiners retained | Retains NOUN, VERB, ADJ, ADV, PROPN only |
| **Vectorisation** | Word TF-IDF (1,2)-grams, 8,000 features | Word TF-IDF 8,000 + Char TF-IDF 3-5-gram 5,000 |
| **Vocab / avg tokens** | 290 unique tokens; 5.31 avg tokens/query | 277 unique tokens; 4.59 avg tokens/query |

### 4.5 Confusion Analysis

Analysis of confusion patterns delineates the structural boundaries of each pipeline. Both pipelines demonstrate the same dominant confusion pairs, suggesting that these pairs represent genuine semantic proximity rather than artefacts introduced by preprocessing.

**login_issue → reset_password:** Both categories relate to authentication failure. Queries such as 'cannot log in after password reset' contain tokens relevant to both classes. Differentiation requires identifying whether the user is experiencing a login failure (login_issue) or requesting a password change (reset_password).

**data_privacy → delete_account**: Both categories involve personal data and account management terminology. The delete-account phrase chunk in Pipeline 2 reduces confusion for queries explicitly mentioning account deletion. However, this method does not resolve confusion for queries addressing data rights without reference to account actions.

**billing_history → payment_issue**: Both categories involve financial transactions. The temporal distinction between past records (billing_history) and current failures (payment_issue) is not directly encoded in bag-of-words features.

**order_tracking → shipping_delay** and vice versa: Both categories share delivery-status vocabulary. The TIME_EXPR tags in Pipeline 2 preserve duration expressions, which are more frequent in shipping_delay and offer only marginal separation. Nevertheless, substantial overlap remains.

These confusions are structurally hard to avoid within the bag-of-words paradigm. They define the performance ceiling for both pipelines and highlight the necessity of a transformer-based approach, as discussed in the Conclusion.

## 5. Conclusion

Both pipelines performed much better than the baseline, showing that the preprocessing steps and the Logistic Regression model are picking up useful patterns from 308 queries across 14 classes. By using the same classifier, we could focus on the impact of preprocessing. Pipeline 2 did better than Pipeline 1 (CV macro F1: 0.915 vs 0.883) and had less variance. This improvement comes from phrase chunking, which creates clear compound features, lemmatisation for consistent tokens, and character n-grams that help handle variation and rare forms.

**The most effective step was converting '2fa' to 'two factor authentication', which led to almost perfect results for the two_factor_auth class, even though there were only 10 samples. Keeping important stop-words like 'cannot' and 'locked' helped keep key** intent differences. Phrase chunking also improved class separation by turning multi-word IT terms into single, strong features.

Numeric tagging, location NER, and POS filtering did not make much difference ($\Delta F1 \approx 0$). This was because these features were rare, not specific to any class, or already handled by earlier cleaning steps.

**There are some limitations. The small dataset makes the results less reliable, especially for classes with fewer samples. Since TF-IDF does not consider word order, it can mix up classes that are similar in meaning.** The model also relies on manually updated abbreviation and phrase lists, which might not work well with new terms.

**Looking ahead, the main priorities are using** domain-adapted transformers for better context, switching to PMI-based phrase detection instead of manual rules, adding more data for minority classes, and using active learning to grow the labeled dataset more efficiently.

**REFERENCES** –


*Core methods / libraries used*

*Bird, S., Klein, E. and Loper, E. (2009) Natural language processing with Python: analyzing text with the Natural Language Toolkit. Sebastopol, CA: O'Reilly Media.*

*Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', Journal of Machine Learning Research, 12, pp. 2825–2830.*

*Honnibal, M. and Montani, I. (2017) spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing. Available at: https://spacy.io (Accessed: 20 March 2026).*

*For TF-IDF / IR theory*

*Spärck Jones, K. (1972) 'A statistical interpretation of term specificity and its application in retrieval', Journal of Documentation, 28(1), pp. 11–21.*

*Jurafsky, D. and Martin, J.H. (2023) Speech and language processing. 3rd edn, draft. Available at: https://web.stanford.edu/~jurafsky/slp3 (Accessed: 20 March 2026).*

*Institution*

*University of Bath (2026) MSc Artificial Intelligence: unit catalogue and assessment guidelines. Bath: University of Bath. Available at: https://www.bath.ac.uk (Accessed: 20 March 2026).*